Assignment 1 – Mini Project Report

Console Record-Management Application (Student Record Management System)

Name: Sneha Sujeesh **Roll No.:** 12 **Programme:** MCA **Institution:** ITM Skills University
Assignment: Assignment 1 – Mini Project **Language:** Python 3

1. Introduction

This project is a menu-driven console application written in Python that manages student records. It lets a user add, view, search, update and delete records, and it stores the data in a file so nothing is lost when the program is closed. The project combines the main Python fundamentals covered in the course into one working application.

2. Objectives

- Build a complete menu-driven console application in Python.
- Apply data types, variables, conditionals, loops and functions in a real program.
- Handle invalid input and file errors using exception handling.
- Use File I/O so that records persist between program runs.
- Keep the code readable, modular and well commented.

3. Tools and Technologies

- **Language:** Python 3
- **Standard libraries:** `json` (storing records), `os` (checking file existence)
- **Storage format:** JSON file (`students.json`)
- **Platform:** macOS Terminal (zsh)
- **Version control:** Git and GitHub

4. Python Concepts Used

4.1 Data Types and Variables

Each student is stored as a dictionary containing `name` (string), `course` (string) and `marks` (float). All students are held in one outer dictionary keyed by student ID (string), which allows fast lookup, duplicate checking and easy deletion.

4.2 Conditional Statements and Loops

`if/elif/else` checks are used for validation, such as checking whether an ID already exists or whether marks fall between 0 and 100. A `while True` loop runs the main menu until the user chooses Exit, and further `while` loops repeat prompts until valid input is given. `for` loops go through the records for viewing and searching.

4.3 Functions

The program is divided into small functions, each with one job:

Function	Purpose
`load_records()`	Reads records from the JSON file
`save_records(records)`	Writes records to the JSON file
`get_non_empty(prompt)`	Validates that text input is not empty
`get_marks(prompt)`	Validates that marks are a number from 0 to 100
`add_record(records)`	Adds a new student
`view_records(records)`	Displays all students
`search_record(records)`	Searches by ID or name
`update_record(records)`	Edits an existing student
`delete_record(records)`	Removes a student after confirmation
`show_menu()`	Prints the menu
`main()`	Runs the program loop

4.4 Exception Handling

- `ValueError` is caught when the user types text instead of a number for marks.
- `json.JSONDecodeError` is caught if the data file is corrupted, and the program starts with empty records instead of crashing.
- `OSError` is caught for file read and write problems.
- `KeyboardInterrupt` and `EOFError` are caught so that pressing Ctrl+C exits cleanly.

4.5 File I/O

Records are loaded from `students.json` when the program starts and saved after every add, update or delete. The `with open(...)` statement is used so files are always closed properly. If the file does not exist, the program begins with an empty record set.

4.6 Menu-driven Design

The main menu offers six options. A dictionary maps each menu choice to its function, which keeps the code short and easy to extend.

5. Program Design

5.1 Menu

```
===== Student Record Management =====
1. Add record
2. View all records
3. Search record
4. Update record
5. Delete record
6. Exit
```

5.2 Program Flow

1. The program starts and loads existing records from `students.json`.
2. The menu is displayed and the user enters a choice.
3. The matching function runs. Add, update and delete save the file immediately.
4. The menu is shown again until the user selects Exit.

5.3 Data Structure

```
{
    "S1": {
        "name": "Asha",
        "course": "MCA",
        "marks": 91.0
    }
}
```

6. Features

- Add a student with a unique ID.
- View all stored records.
- Search by ID or by part of a name (not case sensitive).
- Update any field; pressing Enter keeps the current value.
- Delete with a yes/no confirmation.
- Input validation for empty values and out-of-range marks.
- Persistent storage that survives restarts.
- Graceful handling of a missing or corrupted data file.

7. How to Run

1. Install Python 3 and open Terminal.
2. Go to the project folder: `cd student-record-manager`
3. Run the program: `python3 student_records.py`
4. Choose options from the menu. Records are saved to `students.json` automatically.

8. Sample Input and Output

Adding a record, including invalid marks:

```
Enter your choice (1-6): 1
Enter student ID: S1
Enter name: Asha
Enter course: MCA
Enter marks (0-100): abc
Invalid input. Please enter a number.
Enter marks (0-100): 88
Record added successfully.
```

Viewing records:

```
--- All Records ---
ID: S1 | Name: Asha | Course: MCA | Marks: 88.0
ID: S2 | Name: Rahul | Course: MCA | Marks: 75.0
```

Searching:

```
Enter ID or name to search: asha
ID: S1 | Name: Asha | Course: MCA | Marks: 88.0
```

Deleting:

```
Enter ID of record to delete: S2
Are you sure? (y/n): y
Record deleted.
```

9. Screenshots

(Insert your screenshots here: menu, add record, view records, search, update, delete, invalid input handling, and the saved `students.json` file.)

10. Testing

Test	Input	Expected result	Result
Add valid record	ID S1, marks 88	Record saved	Pass
Duplicate ID	ID S1 again	Rejected with message	Pass
Non-numeric marks	`abc`	Error shown, re-prompted	Pass
Marks out of range	`150`	Error shown, re-prompted	Pass
Search by name	`asha`	Matching record shown	Pass
Update marks	New value 91	Marks changed and saved	Pass
Delete with confirmation	`y`	Record removed	Pass
Invalid menu choice	`9`	Error message, menu repeats	Pass
Restart program	Reopen app	Earlier records still present	Pass
Missing data file	No `students.json`	Starts with empty records	Pass

11. Conclusion

The project meets all the assignment requirements. It uses data types, loops, conditionals, functions, exception handling and File I/O together in a structured, menu-driven application. Working on it showed how to break a program into small functions, validate user input, and keep data safe between runs. Possible future improvements include a CSV export, sorting records by marks, and a simple login system.

12. GitHub Repository

Repository link: *(paste your GitHub repository URL here)*

Repository contents: `student_records.py`, `students.json`, `README.md`, this report, and a `screenshots/` folder.